Root Cause Analysis Engine — Claude Implementation Prompt

Scope: Dimension Analysis + Contribution Analysis + Hierarchical Drill-Down

You are a Senior Data Scientist + AI Engineer working on an existing production-oriented project called:
AI Root Cause Analysis Platform.

IMPLEMENT ONLY these RCA analytical components:
1. Dimension Analysis
2. Contribution Analysis
3. Hierarchical Drill-Down

Do not rebuild Data Foundation. Do not implement Anomaly Detection. Do not implement Ollama/LLM explanations.

1. Existing capabilities — stable dependencies

Already implemented:
- Data ingestion
- Data profiling
- Schema validation
- KPI detection
- KPI selection
- KPI definition
- SQL Server integration
- Parquet dataset storage
- DuckDB analytical access
- Anomaly Detection

Treat these as stable. Reuse them. Do not duplicate or unnecessarily refactor them.

```
Architecture:
Analysis Ready KPI + comparison periods
        ↓
RCA Engine
        ↓
Dimension Analysis
        ↓
Contribution Analysis
        ↓
Hierarchical Drill-Down
        ↓
Structured RCA Result
```

2. Objective

Answer:
"Which parts of the business data contributed most to the KPI change?"

Example:
Revenue
June → $10.2M
July → $8.7M
Change → -$1.5M

Expected result:
Primary Driver: Cairo
Impact: -$1.2M
Contribution: 80%

Drill-down:
Revenue → Cairo → Product A → Enterprise → -$700K

All results must come from the dataset. Never hardcode the example.

Important: this is contribution analysis, not proof of causality. Use terms such as Primary Driver, Secondary Driver, Contributing Factor, and Offsetting Factor.

3. Input

Consume the existing KPI Definition, for example:

```
{
  "name": "Revenue",
  "column": "revenue",
  "aggregation": "SUM",
  "time_column": "date",
  "dimensions": ["region", "product", "customer_segment"],
  "comparison": "previous_period"
}
```

Use the existing dataset, KPI definition, current period, and comparison period. Reuse existing models/services.

4. Overall KPI change

Calculate:
- previous KPI
- current KPI
- absolute change
- percentage change

Respect:
SUM, AVG, COUNT, COUNT_DISTINCT, MIN, MAX, MEDIAN.

Do not assume every KPI is SUM.

5. Dimension Analysis

Question:
"Where did the KPI change happen?"

For every configured dimension:
1. Verify it exists.
2. Group by the dimension.
3. Calculate KPI for previous period.
4. Calculate KPI for current period.
5. Calculate absolute change.
6. Calculate percentage change where valid.
7. Identify positive and negative movements.
8. Rank values by meaningful impact.

Example:

```
Region | Previous | Current | Change
Cairo       4.0M      2.8M     -1.2M
Alexandria  3.0M      3.0M      0
Giza        3.2M      2.9M     -0.3M
```

Use DuckDB for large aggregations. Avoid loading the entire dataset into Pandas.

6. Impact is more important than percentage change

Never rank drivers only by percentage change.

Example:
Product A: $100K → $50K = -50% = -$50K
Product B: $5M → $4M = -20% = -$1M

Product B is the stronger business driver.

Percentage Change != Business Impact.

7. Contribution Analysis

Question:
"How much did each factor contribute to the overall KPI change?"

Example:
Overall change = -$1.5M
Cairo change = -$1.2M

Contribution = (-1.2M / -1.5M) × 100 ≈ 80%

Distinguish:
- negative drivers: move in same direction as the overall change
- offsetting factors: move in the opposite direction

Example:
Overall = -$1.5M
Cairo = -$1.2M
Giza = -$0.3M
Alexandria = +$0.2M

Return primary/secondary negative drivers and offsetting factors separately.

8. Contribution edge cases

Handle:
- overall change = 0 → no division by zero; contribution unavailable/not applicable
- previous KPI = 0 → percentage change must be handled safely
- negative KPI values
- non-additive aggregations

For AVG, MEDIAN, MIN, MAX, do not pretend simple additive contribution percentages are mathematically valid. Use a defensible interpretation or explicitly mark contribution as "Not directly attributable".

Correctness is more important than forcing a percentage.

9. Driver ranking

Rank candidate drivers using:
1. absolute KPI impact
2. direction relative to overall KPI change
3. contribution where valid
4. analytical validity

Do not rank only by percentage change.

10. Hierarchical Drill-Down

Question:
"What happened inside the strongest driver?"

```
Example:
Revenue ↓ $1.5M
→ Region: Cairo ↓ $1.2M
→ Product: Product A ↓ $900K
→ Segment: Enterprise ↓ $700K
```

Final path:
Revenue → Cairo → Product A → Enterprise

The path must be DATA-DRIVEN. Do not hardcode Region → Product → Segment.

If Product explains more than Region, the engine may choose:
Revenue → Product A → Cairo → Enterprise.

11. Drill-down algorithm

Conceptually:

```
KPI Change
  ↓
Analyze all available dimensions
  ↓
Rank candidates
  ↓
Select strongest meaningful driver
  ↓
Filter to that driver
  ↓
Analyze remaining dimensions
  ↓
Rank again
  ↓
Continue
```

Stop when:
- no dimensions remain
- no meaningful driver remains
- impact/contribution is below configured minimum
- maximum drill depth is reached

Use configurable maximum depth. Never allow infinite recursion.

12. RCA tree

Create a typed structured tree similar to:

```
{
  "dimension": "region",
  "value": "Cairo",
  "previous_value": 4000000,
  "current_value": 2800000,
  "absolute_change": -1200000,
  "percentage_change": -30.0,
  "contribution": 80.0,
  "children": [
    {
      "dimension": "product",
      "value": "Product A",
      "previous_value": 2500000,
      "current_value": 1600000,
      "absolute_change": -900000,
      "children": []
    }
  ]
}
```

Follow existing Pydantic conventions and do not add meaningless fields.

13. Null and high-cardinality dimensions

Handle null dimension values explicitly using existing project conventions.

For high-cardinality dimensions, use configurable TOP_K_DRIVERS (for example 10). Do not return thousands of candidates.

14. API

Add the minimum RCA endpoint following existing FastAPI conventions.

```
Possible endpoint:
POST /api/rca/analyze
```

```
Input:
{
  "dataset_id": "...",
  "kpi_definition_id": "...",
  "current_period": "2026-07",
  "comparison_period": "2026-06"
}
```

If comparison periods are already resolved by the KPI definition, reuse that logic.

Return:
- KPI summary
- overall change
- dimension analyses
- contribution analyses
- primary drivers
- secondary drivers
- offsetting factors
- RCA tree

Do not create unnecessary endpoints.

15. Frontend

Implement only the UI needed for this RCA feature.

Show:
- KPI summary: previous/current/change
- Primary Drivers
- Secondary Drivers
- Offsetting Factors
- Hierarchical RCA Tree

```
Example:
Revenue ↓ $1.5M
  ↓
Cairo — $1.2M / 80%
  ↓
```

```
Product A — $900K
  ↓
Enterprise — $700K
```

Use existing Next.js, React, TypeScript, Tailwind, shadcn/ui, and Plotly where useful. Do not redesign unrelated screens.

16. Performance

Use existing Parquet + DuckDB.

Prefer DuckDB for:
- KPI aggregation
- period filtering
- GROUP BY
- dimension analysis
- drill-down queries

Avoid full-dataset Pandas loads and unnecessary repeated scans.

17. Suggested backend responsibilities

Only if compatible with the existing architecture:

```
analytics/rca/
- dimension_analysis.py
- contribution.py
- driver_ranking.py
- drill_down.py
- models.py
```

services/
- rca_service.py

schemas/
- rca.py

api/routes/
- rca.py

Responsibilities:
Dimension Analysis → dimension-level KPI changes
Contribution Analysis → contribution/impact
Driver Ranking → strongest candidates
Drill-Down → recursive investigation
RCA Service → orchestration
API → structured response

Do not blindly create this structure; reuse existing modules.

18. Testing

Create deterministic tests for:
- overall KPI change
- dimension grouping
- previous/current values
- absolute and percentage change
- contribution
- positive/negative contributors
- zero total change
- non-additive KPI handling
- driver ranking
- hierarchical drill-down
- no dimensions
- empty dataset
- null dimensions
- high cardinality
- ties
- no meaningful driver
- maximum depth
- zero previous KPI
- negative KPI

19. Golden test dataset

Use:

```
Period | Region | Product | Segment | Revenue
```

June | Cairo | A | Enterprise | 500
June | Cairo | B | SMB | 300
June | Giza | A | Enterprise | 400
June | Giza | B | SMB | 300

July | Cairo | A | Enterprise | 200
July | Cairo | B | SMB | 300
July | Giza | A | Enterprise | 400
July | Giza | B | SMB | 300

The test should discover:
Revenue → Cairo → Product A → Enterprise

Do not hardcode Cairo/Product A/Enterprise.

20. Do not use the LLM for analytics

Ollama must NOT calculate:
- KPI change
- dimension impact
- contribution
- driver ranking
- drill-down

The backend performs deterministic analytical calculations.

Future:
Data → RCA Engine → Structured RCA Result → Ollama → Business Explanation

This phase only produces structured evidence.

21. Definition of Done

- Analysis Ready KPI can be submitted.
- Overall KPI change is correct.
- Every configured dimension can be analyzed.
- Drivers are ranked by meaningful impact.
- Contribution is calculated where mathematically valid.
- Positive/negative contributors are distinguished.
- Primary and secondary drivers are identified.
- Offsetting factors are identified.
- Hierarchical drill-down works.
- Drill-down is data-driven.
- Maximum depth is enforced.
- High-cardinality dimensions are controlled.
- Null dimensions are handled.
- Non-additive aggregations are handled correctly.
- Typed API response exists.
- Frontend visualizes the result.
- Tests pass.
- Golden dataset test passes.
- Phase 1 and Anomaly Detection still work.
- No Ollama/LLM logic is added.
- No unrelated feature is added.

22. Execution rules

Before coding:
1. Inspect the repository.
2. Identify KPI Definition and comparison-period logic.
3. Identify Parquet/DuckDB access.
4. Identify existing service/schema/API conventions.
5. Identify any existing RCA demo code.
6. Identify the minimum files required.
7. Produce a short implementation plan.

8. Implement the backend.
9. Implement the API.
10. Implement the frontend.
11. Add tests.
12. Run the full test suite.
13. Verify Phase 1 and Anomaly Detection are not broken.

At the end report:
1. Files created
2. Files modified
3. Dimension Analysis implementation
4. Contribution strategy
5. Driver ranking strategy
6. Drill-down strategy
7. API endpoint
8. Frontend components
9. Tests added
10. Tests passed
11. Limitations/technical decisions

Do not implement anything outside Dimension Analysis, Contribution Analysis, Hierarchical Drill-Down, and the minimum API/frontend integration required to expose them.